CS 61A — Week 1

August 24–30: Welcome, Functions; CP 1.1–1.3; Discussion 0, Lab 0, HW 1

§1 Evaluate a call from its parts

The question. What value does `mul(add(4, mul(4, 6)), add(3, 5))` produce? Which computation happens first?

`from operator import add, mul` binds `add` to addition and `mul` to multiplication. An **expression** describes a computation and evaluates to a value. A numeral such as `2` evaluates to its number; a name such as `pi` evaluates to its current binding; a string such as `'are you human'` is text. Functions are also values. In `add(1 + 1, 3)`, `add` is the **operator expression**; `1 + 1` and `3` are the **operand expressions**. Their values are the function and its arguments.

Proposition (Call rule — Lectures 1–2; CP 1.2)

Evaluate the operator first, then the operands from left to right, then apply the resulting function to the argument values. Apply the same rule inside each nested call.

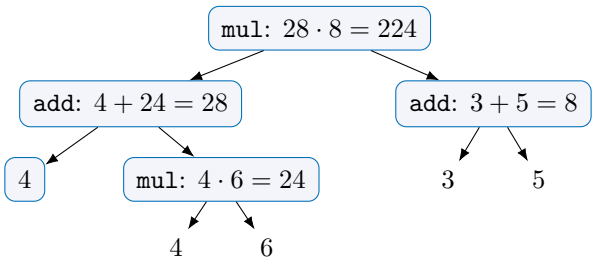
Where the proof comes from

Want. Determine a call's value without guessing the order of its inner computations.

Know. A function needs argument *values*, not unevaluated expressions.

Key step. Finish each operand before applying its enclosing function. Separately evaluate the operator: its name might have been rebound.

1. Look up the outer `mul`; then start its first operand, `add(4, mul(4, 6))`.
2. Look up this `add`, evaluate `4`, then look up the inner `mul` and evaluate `4`, `6`. Apply it to obtain `24`; the first `add` returns `28`.
3. Evaluate the second operand: `add(3, 5)` returns `8`. Apply the outer `mul` to `28` and `8`: the result is `224`.



A parent call uses the values of its children. Evaluation visits the operator first and the operands from left to right; the displayed arithmetic happens after the operands finish.

Example (The reading's similar tree has a different first numeral)

CP 1.2 uses `mul(add(2, mul(4, 6)), add(3, 5))`. Its first argument is $2 + 24 = 26$, so it returns $26 \cdot 8 = 208$, not 224.

Arithmetic and library calls — CP 1.2–1.3. `from math import sqrt, pi` binds a square-root function and a constant; `from operator import add, sub, mul, truediv, floordiv` binds functions for arithmetic. Function-call notation permits several arguments, arbitrary nesting, and ordinary keyboard symbols.

Expression	Value and reason
`42; -1 - -1`	42; 0 (subtracting −1 adds 1)
`1/2 + 1/4 + 1/8 + 1/16 + 1/32 + 1/64 + 1/128`	$127/128 = 0.9921875$
`max(7.5, 9.5); max(1, -2, 3, -4)`	9.5; 3
`pow(100, 2)`	10000
`max(min(1, -2), min(pow(3, 5), -4))`	$\max(-2, -4) = -2$
`sqrt(256); add(14, 28)`	`16.0`; 42
`sub(100, mul(7, add(8, 4)))`	$100 - 7 \cdot 12 = 16$
`2 + 3 * 4 + 5; (2 + 3) * (4 + 5)`	19; 45 (parentheses change grouping)
`5 / 4; 8 / 4; 5 // 4; -5 // 4`	`1.25`; `2.0`; 1; −2

Python integers also represent large exact values: `pow(2, 100)` returns 1267650600228229401496703205376. `/` performs true division; `//` rounds the quotient down, including for negative inputs. `truediv(5,4)` and `floordiv(5,4)` give `1.25` and `1`.

Shortcuts

- Replace a completed inner call by its value, keeping the surrounding expression intact: `max(min(1,-2), -4)` becomes `max(-2,-4)`.
- Powers use `**`, not `^`. Multiplication precedes addition; calls evaluate their arguments before applying the function.

Traps

- "The innermost multiplication happens first" omits name lookup. In the lecture tree, the *first expression evaluated* is the outer operator `mul`.
- `-5 // 4` is −2, not −1: floor division rounds toward $-\infty$.

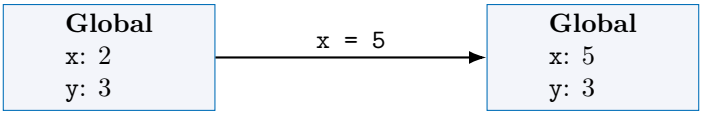
§2 Names store values, not formulas

The question. After `x = 2`, `y = x + 1`, `x = 5`, is `y` now 6?

A **binding** associates a name with a value. A **frame** holds bindings, with at most one current value per name. An **environment** is an ordered sequence of frames used to look up names. Top-level assignments use the global frame.

Proposition (Assignment — Lecture 2; CP 1.2)
Evaluate all right-hand expressions using the old bindings; then bind the names on the left to the resulting values. Reassignment replaces a binding. Assignment is a statement, not an expression that returns a value.

1. `y = x + 1` evaluates $2 + 1$ once and binds `y` to 3.
2. `x = 5` changes only `x`. Nothing reevaluates the earlier expression; `y` stays 3.
3. Likewise, from `x, y = 3, 4.5`, the statement `y, x = x, y` first obtains $(3, 4.5)$, then sets `y` to 3 and `x` to 4.5.



Rebinding `x` replaces its value; it creates no instruction to recompute `y`.

Example (Circle quantities — CP 1.2)

With `radius = 10`, `diameter = 2 * radius` is 20; `area = pi * radius * radius` is approximately 314.1592654, and `circumference = 2 * pi * radius` is approximately 62.8318531. Changing `radius` to 11 leaves those stored values unchanged. Reassigning `area = pi * radius * radius` computes the new value 380.1327111. The reading also evaluates `pi * 71 / 223`, approximately 1.0002380; CP 1.3 binds `tau = 2 * pi`, approximately 6.2831853.

Functions can have several names. Each row below starts with the standard built-in bindings.

Statements, then expression	Outcome
`f = max; f(2, 3, 4)`	4: `f` refers to the maximum function.
`f = max; f = 2; f`	2: only `f` was rebound.
`f = max; max = 3; f(2,3,4)`	4: the saved function is unchanged.
`max = 5; max(1, 2)`	`TypeError`: the operator now evaluates to a number.
`max = pow; max(2, 5 + 5)`	1024: the name now refers to exponentiation.
`x = 2; x = x + 1; x`	3: the right side uses the old 2.

The lecture demo also sets `max = pow`, then `pow = max`: both names still refer to exponentiation, so `pow(2, 10)` remains 1024.

Shortcuts

- For a name, look up its current value; do not infer behavior from the spelling. This explains `max = pow` without a special case.
- For a multiple assignment, write down the entire right-hand tuple of values first. This makes swaps safe without a temporary name.

Traps

- `x = x + 1` is a valid assignment, not an impossible algebraic equation. The old and new bindings occur at different times.
- `1 + 2 = x` and `x - 1 = 2` are invalid assignment targets; assignment does not solve equations.

§3 A function call gets its own local bindings

The question. Why does `square(square(3))` return 81? Why does a local `x` not overwrite the global `x`?

```
1 from operator import mul
2
3 def square(x):
4     return mul(x, x)
5
6 def sum_squares(x, y):
7     return square(x) + square(y)
```

A **def statement** creates a function and binds its name; it does not execute the body. Its header gives a name and **formal parameters**; together they describe its signature. Calling it supplies **arguments**. The number of arguments must match this signature: `square()` and `square(2,3)` raise `TypeError`. The indented body executes only when the function is applied. A `return` sends its expression's value back to the caller and ends that call.

Proposition (Application and lookup — Lecture 2; CP 1.3)

After evaluating a call's operator and arguments, create a new local frame, bind its parameters to the arguments, and execute its body there. For the globally defined functions here, look up a name in that local frame first, then the global frame (then Python's built-ins). A local assignment changes only that local frame.

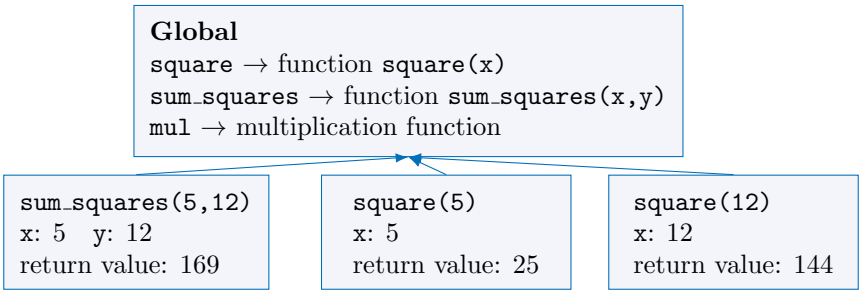
Where the proof comes from

Want. Reuse the same function with different inputs without changing earlier calls or the caller's variables.

Know. The body refers to parameter names such as `x`; the caller supplies values such as 5 or 12.

Key step. Give each call a separate table of bindings. To evaluate `square(5)`, bind its own `x` to 5; do not replace every `x` in the program.

1. `sum_squares(5, 12)` creates a local frame with `x = 5`, `y = 12`.
2. Its first operand calls `square(5)`. A new frame binds `x = 5`; `mul(5,5)` returns 25.
3. Its second operand calls `square(12)` in another new frame: `x = 12`, result 144. The enclosing call returns $25 + 144 = 169$.



Each arrow identifies the next frame for name lookup. Both `square` calls look up `mul` globally; they do not search the `sum_squares` frame. Return values are annotations, not bindings.

Nested calls and separate local names. `square(21)` gives 441; `square(add(2,5))` first obtains 7 and then 49; `square(square(3))` computes 9, then 81. `sum_squares(3,4)` returns $9 + 16 = 25$. `square(-2)` binds its local `x` to -2 and returns 4.

Lecture 2's local assignment.

```
def g(y):
    x = 2 * y
    return x + 1

x = 2
first = g(x)
second = g(3 * x) + 3
y = 3
third = g(y)
```

Call	argument	local y	local x	returned value
`g(x)`	2	2	4	5
`g(3 * x)`	6	6	12	13
`g(y)`	3	3	6	7

Thus `first = 5`, `second = 16`, `third = 7`; global `x` stays 2 and global `y` stays 3.

A parameter can have the function's name — Lecture 2.

```
from operator import mul

def square(square):
    return mul(square, square)

answer = square(3)
```

The operator `square` is looked up *before* entering the call, so it finds the function globally. Inside, the local name `square` means 3. The result is 9; the global function binding is unchanged.

Functions as abstractions — CP 1.3. A function's **domain** describes allowed inputs; its **range** describes possible outputs; its intent describes their relationship. Once `square` is known to return the square of an input, `sum_squares` can use that relationship without depending on how the multiplication is implemented. Consistently renaming `square`'s parameter preserves its behavior. For integer arithmetic, `mul(x,x)` and `mul(x,x-1) + x` give the same square because $x(x-1)+x = x^2$; this identity need not hold bit-for-bit for floating-point arithmetic.

Scope is where a name can refer to a binding in the code; an environment is the runtime sequence of bindings used by a particular evaluation. Use lowercase names with underscores, choose names for their role, and avoid confusing names such as `l`, `O`, `I`. A function's **intrinsic name** is the name recorded when it is created; binding another name to it does not change that recorded name. The reading's `g()` returning 1, reassignment `g = 2`, then redefinition `g(h,i)` returning `h+i` illustrate three different successive bindings; the final `g(1,2)` returns 3.

Shortcuts

- Track the frame and the name together: "local `x` in the second `square` call" identifies a value that bare "`x`" does not.
- Evaluate the argument *before* binding the parameter: in `g(3*x)`, global `x = 2` produces argument 6.

Traps

- The caller's frame is not automatically the next place to look for names. These globally defined functions look up missing local names globally.
- Defining `square` does not compute a square yet. Calling it does. A returned value is not an additional variable in the local frame.

§4 Printed output and returned values are different

The question. What is printed by `print(print(1), print(2))`? What value does that whole call return?

Proposition (Pure functions, effects, and `None` — CP 1.2; Lecture 2)

A pure function returns the same result for the same argument values, without observable side effects. Printing is a side effect: `print` displays its arguments and returns the special value `None`. A function that reaches the end of its body without executing `return` also returns `None`.

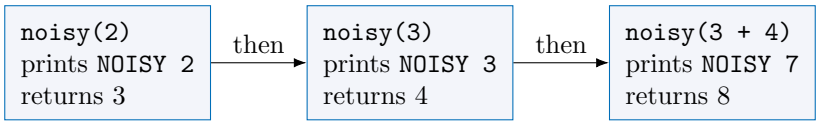
1. Evaluate outer `print`; then evaluate `print(1)`: display `1`, return `None`.
2. Evaluate `print(2)`: display `2`, return `None`.
3. The outer function therefore receives `None, None`; it displays `None None` and returns `None`. The interactive interpreter suppresses automatic display of that last value.

Example (CP 1.2)

`abs(-2)` returns 2 without printing. `print(1,2,3)` displays `1 2 3` and returns `None`. Executing `y = print(-3)` displays `-3` and binds `y` to `None`; `print(y)` then displays `None`. Evaluating `print(-3) * 2` first displays `-3`, then raises `TypeError`: multiplying `None` by 2 is undefined.

Lecture 2's noisy calls.

```
1 def triple(x):
2     return 3 * x
3
4 def noisy(x):
5     print('NOISY', x)
6     return x + 1
7
8 result = noisy(noisy(2) + noisy(3))
```



The outer call waits for $3 + 4$ to be computed. Only `NOISY` lines are printed by this assignment; `result` is bound to 8.

At the prompt, evaluating the same call without assignment also displays the final value `8`. A script does not automatically display an expression's value. `triple(4)` returns 12; replacing its return with `print(x)` would print 4 and return `None`.

Shortcuts

- Keep two separate records: the output lines produced so far, and the value returned by each completed call. Add a line only when `print` runs (or the prompt displays a non-`None` result).

Traps

- `print(print(1), print(2))` displays three lines, not one: `1`, then `2`, then `None None`.
- A body containing only `mul(x,x)` discards its value. Use `return mul(x,x)` to return it; printing it does not substitute for returning it.

§5 Choosing and repeating operations: tools for HW 1

The question. How can one function choose addition or subtraction, and another repeat divisibility tests until it finds a factor?

Comparisons `<`, `>`, `<=`, `>=`, `==`, `!=` produce `True` or `False`. `=` assigns; `==` compares. A compound statement has a header ending in a colon and an indented body. Statements in a body execute in order until control redirects them, for example by `return`.

Proposition (The rules needed by HW 1 — CP 1.5)

For `if`/`elif`/`else`, test conditions in order, execute only the first true clause's body, and skip the remaining clauses; execute `else` if no condition is true. For `while`, test its condition before each iteration; if true, execute the whole body and test again; if false, continue after the loop.

A loop updates bindings in the current call's frame; it does not create a new call frame for each iteration. A **remainder** is produced by `%`. For integers n and $d > 0$, $n = (n//d)d + (n\%d)$ with $0 \le n\%d < d$. For positive integers, `n % d == 0` means d divides n; `n // d` then gives the exact integer quotient. In particular, `n % 2 == 0` tests evenness. A return inside a loop ends the function immediately. A return after the loop waits for a false condition.

Example (Choose an operation, then call it — HW 1 Q1)

When $b < 0$, $a + |b| = a - b$, so select `sub`; otherwise select `add`. The selected value must be a *function*. For $a = -1, b = -4$, the eventual call is `sub(-1,-4)`, returning 3. Writing `sub(a,b)` in the selection step would select a number instead.

Where the proof comes from

Want. Find the largest proper factor of $n > 1$ without listing every factor.

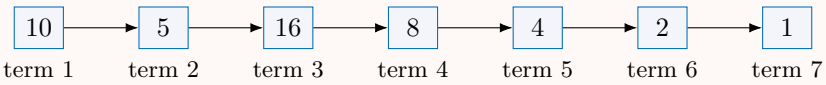
Know. Factors occur in pairs: $d \mid n$ implies $n/d \mid n$.

Key step. The smallest divisor $d \ge 2$ gives the largest partner $n/d < n$. Test $d = 2, 3, \ldots$ and stop at the first divisor. The test must eventually succeed at $d = n$.

1. Any proper factor $q < n$ has partner $n/q \ge 2$. The smallest tested divisor d satisfies $d \le n/q$, so $q \le n/d$.
2. Thus n/d is at least every proper factor, and is itself a proper factor. For $n = 15$, test $d = 2$ (remainder 1), then $d = 3$ (remainder 0); return $15//3 = 5$.
3. For 80, the first divisor is 2, returning 40. For prime 13, the first is 13, returning 1. A descending search from $n - 1$ to 1 is another valid method: the first divisor encountered is largest.

Example (Counting printed terms — HW 1 Q4)

Start with positive integer n. If even, replace it by $n//2$; if odd and not 1, replace it by $3n + 1$. Stop at 1.



Six replacements produce seven printed terms. Print the initial term too, and count it before testing whether another replacement is needed.

For input 1, print 1 and return length 1. For input 27, there are 112 terms and the largest is 9232. Termination for *every* positive integer is the unproved Collatz conjecture; these examples do not prove it.

Shortcuts

- A loop table should record values *before* the test, the test result, and the update. For 15, $d = 2 \to 3$ changes a failed divisibility test into a successful one.
- If a loop counts terms including its starting term, initialize the count to 1; each replacement increases it by 1.

Traps

- Changing a variable in a loop does not by itself prove termination: `n = n + 1` inside `while n > 0` never reaches a false condition when started positive.
- Using `/` for hailstone division changes integers to floating-point values (for example `4.0`); use `//` for the assignment's integer sequence.

§6 Build an expression by choosing its last operation

The question. With only 5, parentheses, and the functions below, how can a short expression evaluate to 2026?

Discussion 0 Q3 defines $f(x) = x + 1$, $g(x) = 2x - 1$, and $h(x, y)$ as concatenation of two *different positive integers*. Thus $h(789, 12) = 78912$, while $h(12, 789) = 12789$. Only `f`, `g`, `h`, `5` and parentheses may be repeated; commas separate the two arguments of `h`.

```
def f(x):
    return x + 1

def g(x):
    return 2 * x - 1

def h(x, y):
    return int(str(x) + str(y))
```

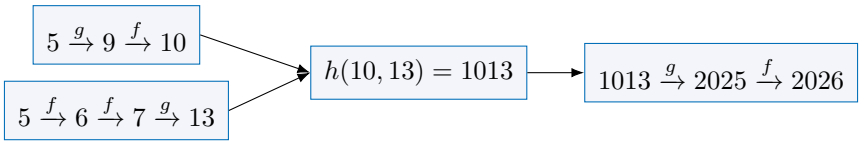
Here `str` converts a number to its decimal text, `+` joins two texts, and `int` converts the joined text back to an integer. Check positivity and inequality before using `h`; its code alone does not enforce them.

Example (Lecture 2's eight-call expression)

Try $2026 = f(2025) = f(g(1013))$. Concatenation gives $1013 = h(10, 13)$. Produce $10 = f(g(5))$ and $13 = g(f(f(5)))$. Combining these gives

$$\texttt{f(g(h(f(g(5)), g(f(f(5))))))} = 2026.$$

Its call count is $1 + 1 + 1 + 2 + 3 = 8$. The discussion's example is `h(f(g(5)), f(f(5)))`: the arguments are 10 and 7, giving 107.



Both input expressions must finish before concatenation. Working backward from 2026 suggests the intermediate values 2025, 1013, 10 and 13.

Why searching by call count can certify minimality.

1. An expression with $n > 0$ calls ends in either `f`/`g` with an $(n-1)$-call operand, or `h` with operand counts k and $n - 1 - k$ for some $0 \le k < n$. These cases include every possible last call.
2. Start with the only zero-call expression, `5`; use the cases above to construct all possible values for each successive call count. The exhaustive check finds no 2026 with 0–7 calls; 8 calls produces the example above.
3. Call count and character count differ. With n calls and b binary calls, there are $b + 1$ leaves `5`, so the written length is $3n + b + (b + 1) = 3n + 2b + 1$ characters, including commas. The solutions separately verify minimum character length for this example.

Traps

- The lecture's search demo allows equal arguments to `h`; the discussion's stated rule forbids them. Filter equal arguments when checking the discussion problem. For example, `h(5,5)` is not permitted.
- Finding one expression proves it works, not that it is shortest. Minimality needs the exhaustive argument above. The lecture's final demo slide also has a stale 2025 target; the worksheet asks for 2026.

§7 Run, inspect, and test the program

The question. The file was edited, but Python still behaves as before. Which program and which saved file are running?

CP 1.1. Computer science studies representations of computations, their logical execution, and abstraction. Python's interpreter executes instructions. Debug by testing a small part, isolating a failing example, and checking each intermediate assumption before changing the code; ask for help with the concrete failure.

Lab 0 setup and execution. Use Python 3.12 or newer and open the extracted `lab00` folder in VS Code with the Python and Provenance extensions. A terminal *shell* accepts these commands; the Python prompt `>>>` instead accepts Python expressions and statements:

```
python3 lab00.py
python3 -i lab00.py
python3 -m pytest -k python_basics --unlock
python3 -m pytest -k twenty_twenty_six
```

The first runs the file; the second runs it and then opens an interactive session with its definitions. The last two check the lab questions. For HW 1, use `python3 -m pytest -k a_plus_abs_b`, replacing the name with `two_of_three`, `largest_factor`, or `hailstone` for the other questions. Use `exit()` to leave Python; Control-C interrupts a running computation.

A **docstring** is a string placed first in a function body to document behavior; `#` begins an interpreter-ignored comment. Neither replaces `return`. A **doctest** shows inputs after `>>>` and expected outputs below. The lab's prompt checks give 2000 and 2026. Passing examples checks those inputs, not every input.

Shortcuts

- Give each function one stated job and test it on ordinary and boundary inputs. `hailstone(1)` checks a loop that runs zero times.
- The prompt automatically displays non-`None` results; a script needs an explicit `print` for output.

Traps

- An old session or wrong folder may run different code. Check the folder, save edits, and restart the session to load changes.
- Submit the ZIP created by `Provenance: Prepare Submission Bundle` in VS Code's command palette to Gradescope; the starter ZIP is not a submission bundle. HW 1 also requires its course checkoff.

Reading attribution. Adapted examples from John DeNero and contributors, *Composing Programs*, third edition, sections 1.1–1.3 (CC BY-NC-SA 4.0). HW 1 also links sections 1.4–1.5, scheduled with Week 2; the control rules needed for its problems are introduced above.

§8 Problem set

Lab 0 Q1–Q2: Python Basics; Twenty Twenty Six. Predict the prompt results for `2000` and `2000 + 26`. Fill only the return expression of `twenty_twenty_six()` so that it returns 2026, using numbers and arithmetic operators (the basic operators are `+`, `*`, `-`; `**` and `%` are also allowed).

Hint: Build a numeric expression; return its value, rather than printing it.

Discussion 0 Q1–Q2: Name Game; Partitions. For Q1, order the group by birthdays starting today; each turn states one's name and hometown, then repeats prior participants from most recent to earliest. For Q2, split into equal halves (sizes may differ by one); spend ten minutes finding a fact shared by all in your half and none in the other half. Facts must be neither visible traits nor preferences. Play two rounds with different splits; finally find a fact shared by the whole group that you think the TA does not share.

Hint: These are group activities, not programming questions; carry out the stated ordering and membership checks.

Discussion 0 Q3: Small Expressions. Using only the allowed functions, 5, and parentheses, find the shortest expression you can that evaluates to 2026. Both inputs to `h` must be different positive integers.

Hint: Try an outer `f`, then an outer `g`, then split decimal digits using `h`. Distinguish a working expression from a minimality proof.

HW 1 Q1: A Plus Abs B. Return $a + |b|$ without calling `abs`. Fill only the two blanks; leave every other statement unchanged.

```
from operator import add, sub

def a_plus_abs_b(a, b):
    if b < 0:
        f = _____
    else:
        f = _____
    return f(a, b)
```

Tests: $(2, 3) \mapsto 5$, $(2, -3) \mapsto 5$, $(-1, 4) \mapsto 3$, $(-1, -4) \mapsto 3$.

Hint: Select a function value using the sign of b.

HW 1 Q2: Two of Three. For positive inputs i, j, k, return the sum of the squares of the two smallest values. Use a single return line in the function body.

```
def two_of_three(i, j, k):
    return _____
```

Tests: $(1, 2, 3) \mapsto 5$, $(5, 3, 1) \mapsto 10$, $(10, 2, 8) \mapsto 68$, $(5, 5, 5) \mapsto 50$.

Hint: Compare the three pairwise sums of squares. For positive inputs, the smallest pair gives the smallest sum; repeated equal values still count separately.

HW 1 Q3: Largest Factor. For integer $n > 1$, return the largest integer smaller than n that divides n evenly. Tests: $15 \mapsto 5$, $80 \mapsto 40$, $13 \mapsto 1$.

Hint: Either test candidates downward from $n - 1$, or find the smallest divisor at least 2 and return its partner.

HW 1 Q4: Hailstone. For positive integer n, print every term of its hailstone sequence, including the starting value and final 1, and return the number of printed terms. Test 10 (seven terms, shown above), 1 (one term), and the assignment's suggested 27.

Hint: Print and count the initial value before a loop that replaces it while `n != 1`. There is no known proof of termination for all inputs.

Past exam: Fall 2025 Midterm 1, Q1(a), p. 3 (excerpt). Execute these assignments, then predict every printed line and the final returned value of the expression below. The original question asks for displayed output; unrelated function definitions are omitted here.

```
about = 6 - 7
about, face = 6, about * 7
print(print(face))
```

Hint: Evaluate both right-hand expressions before rebinding, then separate printing from return values.